

MNIST DIGIT CLASSIFICATION USING TENSORFLOW/KERAS

Deep Learning Assignment Report

1. Introduction

The MNIST (Modified National Institute of Standards and Technology) dataset is one of the most widely used benchmark datasets for handwritten digit recognition. It contains grayscale images representing ten digit classes, from 0 to 9. Each image has a resolution of 28×28 pixels. The objective of this assignment is to develop a simple artificial neural network using TensorFlow/Keras that can learn visual patterns from handwritten digits and classify unseen images into the correct digit class.

The complete workflow includes loading and exploring the dataset, preprocessing the pixel values, designing the neural network, compiling and training the model, evaluating its performance, visualizing accuracy and loss, testing predictions on five images, and performing an experiment by adding dropout layers. This workflow demonstrates the basic steps involved in a supervised deep learning classification problem.

2. Dataset Description and Preprocessing

The MNIST dataset contains 70,000 grayscale images in total. It is divided into 60,000 training images and 10,000 testing images. Every image is represented as a 28×28 matrix of pixel values. Before preprocessing, pixel values range from 0 to 255, where the values represent different grayscale intensities.

For preprocessing, the image arrays are converted to floating-point values and normalized by dividing each pixel by 255. Consequently, the input range changes from 0–255 to 0–1. Normalization provides a consistent numerical scale and generally makes optimization more stable and efficient.

- Training images: 60,000
- Testing images: 10,000
- Image dimensions: 28×28 pixels
- Number of classes: 10 (digits 0–9)
- Original pixel range: 0–255
- Normalized pixel range: 0–1

3. Neural Network Architecture

A feed-forward neural network was implemented using the Keras Sequential API. The first layer is a Flatten layer that converts each 28×28 image into a one-dimensional vector containing 784 input values. The first Dense layer contains 128 neurons and uses the ReLU activation function. The second Dense layer contains 64 neurons and also uses ReLU. The final Dense layer contains 10 neurons with Softmax activation, producing a probability distribution over the ten digit classes.

Layer	Configuration	Activation	Purpose
Input / Flatten	$28 \times 28 \rightarrow 784$	None	Convert image to vector
Dense 1	128 neurons	ReLU	Learn digit features
Dense 2	64 neurons	ReLU	Learn higher-level patterns
Output	10 neurons	Softmax	Predict digit 0–9

The model is compiled using the Adam optimizer, sparse categorical crossentropy loss, and accuracy as the evaluation metric. Adam updates the network weights during training. Sparse categorical crossentropy is suitable because the target labels are integer class values from 0 to 9.

4. Model Training

The model is trained for 10 epochs with a batch size of 32. Twenty percent of the training data is reserved for validation using `validation_split=0.2`. During each epoch, the model learns from the training examples and its performance is monitored on the validation subset. Training accuracy and validation accuracy are recorded, along with training and validation loss.

- Optimizer: Adam
- Loss function: Sparse Categorical Crossentropy
- Metric: Accuracy
- Epochs: 10
- Batch size: 32
- Validation split: 20%

5. Model Evaluation and Prediction

After training, the original model is evaluated on the separate MNIST test set. The test set is not used for weight updates, so it provides an independent measure of how well the trained model generalizes to unseen examples. The exact test accuracy should be copied from the executed notebook output because small differences can occur between training runs.

The model is also tested on five handwritten digit images from the test dataset. For each image, the predicted class is obtained by selecting the output neuron with the highest Softmax probability. The actual label and predicted label are then displayed together. Correct predictions indicate successful classification, while any mismatch identifies an example that the model found difficult.

6. Visualization of Training Performance

Training and validation accuracy are plotted against the number of epochs to show how classification performance changes during learning. Training and validation loss are also plotted. A generally decreasing loss and increasing accuracy indicate that the network is learning useful patterns. The two curves can also help identify possible overfitting if training performance continues improving while validation performance stops improving or begins to deteriorate.

The assignment also includes a model accuracy comparison plot between the original network and the modified dropout network. The plotted values must be taken directly from the variables `test_accuracy` and `experiment_accuracy` generated during notebook execution.

7. Experiment – Adding Dropout

For the required experiment, dropout layers with a rate of 0.2 are inserted after the two hidden Dense layers. Dropout randomly deactivates a fraction of neurons during training. This prevents the network from relying too heavily on particular neurons and can reduce overfitting.

The experimental model uses the same MNIST data, optimizer, loss function, batch size, validation split, and number of epochs as the original model. This makes the comparison more meaningful because the primary architectural change is the addition of dropout.

- Original model: Flatten → Dense(128, ReLU) → Dense(64, ReLU) → Dense(10, Softmax)

- Experimental model: Flatten → Dense(128, ReLU) → Dropout(0.2) → Dense(64, ReLU) → Dropout(0.2) → Dense(10, Softmax)

8. Results and Discussion

The original neural network provides a strong baseline for the MNIST classification task because the dataset is relatively simple and the images have a consistent size. The accuracy and loss graphs provide information about the learning process, while the five-image prediction test provides a direct visual comparison between actual and predicted labels.

The dropout experiment demonstrates how changing the network architecture can affect generalization. The numerical comparison should be interpreted using the actual values produced by the notebook rather than assuming that dropout will always increase accuracy. Depending on the training run, dropout may produce a similar, slightly higher, or slightly lower test accuracy. Its primary purpose in this experiment is to study the effect of regularization.

9. Conclusion

The MNIST handwritten digit classification problem was successfully implemented using TensorFlow/Keras. The dataset was loaded and explored, image pixels were normalized, and a feed-forward neural network was designed with two hidden Dense layers and a ten-neuron Softmax output layer. The model was trained using Adam and sparse categorical crossentropy and evaluated on the independent test dataset.

Training and validation accuracy and loss were visualized to understand the learning process. Five test images were classified and their actual and predicted labels were compared. Finally, dropout was introduced as an experiment to study the effect of regularization. The assignment demonstrates the fundamental deep learning pipeline of data preparation, model construction, training, evaluation, visualization, prediction, and experimentation.